Velora Agent Handbook

Purpose

This document is the operational guide for AI coding agents working on **Velora**, including **VS Code GitHub Copilot**, **Cursor**, and similar tools.

Velora is a **motion-native frontend platform** focused on premium, cinematic interfaces using **semantic HTML** and **modern CSS**, with **zero JavaScript for animations** and the minimum possible JavaScript for accessibility or progressive enhancement.

This handbook exists to keep all agents aligned on:

- product thesis
- non-negotiable architectural rules
- naming conventions
- implementation priorities
- attribute system contracts
- repository structure
- safe prompting patterns
- what agents must never do

---

1. Product Thesis

Velora is not a generic UI kit and not a JavaScript animation wrapper.

Velora is a **declarative motion system for HTML**, powered by modern CSS.

The core idea is:

1. Developers write semantic HTML.
2. They add Velora classes and `vl-*` attributes.
3. Velora maps those contracts to CSS-based layout, motion, scenes, and transitions.

Example direction:

```
<section
  vl-scene="hero"
  vl-timeline="view"
  vl-effect="scene-hero-reveal"
  vl-range="entry 0% cover 70%"
  vl-pin>
  <h1>Build motion-rich interfaces with HTML and CSS</h1>
  <p>No animation libraries. No framework runtime.</p>
</section>
```

The long-term vision includes a visual builder and AI-assisted generation, but the current scope is the **core code-first framework**.

---

2. Non-Negotiable Rules

All agents must follow these rules.

2.1 No JS frameworks in core

Do **not** introduce React, Vue, Framer Motion, GSAP, Locomotive Scroll, Swup, or any comparable runtime as part of the Velora core.

2.2 No JavaScript animation logic in core motion

Do **not** implement animation timing, scroll choreography, reveal effects, or page transitions with JS if modern CSS can handle it.

Preferred platform features include:

- `@view-transition`
- `view-transition-name`
- `view-timeline`
- `scroll-timeline`
- `animation-range`
- `@starting-style`
- `transition-behavior: allow-discrete`
- container queries
- `interpolate-size: allow-keywords`
- logical properties
- `:has()` where appropriate

2.3 HTML first

Agents should preserve semantic HTML and avoid wrapper-heavy markup unless structurally necessary.

2.4 CSS-first architecture

Velora should be implemented as layered CSS with a coherent token system and a stable public API.

2.5 Minimal runtime only when truly needed

JavaScript may be used only for:

- theme persistence
- accessibility helpers where HTML alone is insufficient
- tiny enhancement modules
- future optional runtime hooks

JS is additive, never the foundation of the motion engine.

2.6 All styling contracts must be namespaced

- classes: `.vl-*`
- custom properties: `--vl-*`
- attributes: `vl-*`

2.7 Design tokens use modern color primitives

Prefer `oklch()` and tokenized semantic variables.

2.8 Accessibility is mandatory

Every feature must account for:

- semantic structure
- keyboard access
- visible focus states
- reduced motion
- content readability
- progressive enhancement

---

3. Technical Direction

3.1 Primary stack

- PNPM workspaces
- Turborepo
- Vite
- Astro for docs
- modern CSS as primary implementation layer
- TypeScript only for limited runtime modules

3.2 Monorepo structure

```
velora/
  apps/
    docs/
    playground/
  packages/
    css/
```

3.3 Current strategic focus

The current focus is **v0.1**, which should prove the core framework through:

- foundations and tokens
- layout primitives
- motion primitives
- scene primitives

- page transitions
- premium examples
- documentation

---

4. CSS Architecture

Velora CSS should be organized through cascade layers.

Required order

```
@layer velora.reset, velora.tokens, velora.layout, velora.motion,
velora.components, velora.transitions, velora.utilities, velora.overrides;
```

Layer responsibilities

`velora.reset`

- reset / normalize
- base browser behavior adjustments
- `interpolate-size: allow-keywords`

`velora.tokens`

- colors
- spacing
- typography
- radius
- shadows
- z-index
- motion tokens
- transition tokens

`velora.layout`

- containers
- stack
- cluster
- grid
- sidebar
- scene wrappers
- sticky structural helpers

`velora.motion`

- effect presets
- scene presets
- timeline bindings
- stagger rules
- reveal systems

- hover systems

`velora.components`

- cards
- buttons
- nav
- fields
- dialog
- accordion
- drawer
- tabs

`velora.transitions`

- page transitions
- shared element rules
- view transition styling

`velora.utilities`

- low-level opt-in utility classes

`velora.overrides`

- last-mile overrides only

---

5. Public API Model

Velora should expose a small, stable public API.

5.1 Classes

Use classes for reusable layout and component primitives.

Examples: - `.vl-container` - `.vl-grid` - `.vl-stack` - `.vl-card` - `.vl-button`

5.2 Attributes

Use `vl-*` attributes for motion, scene, and transition intent.

Core v0.1 attribute set:

- `vl-scene`
- `vl-effect`
- `vl-timeline`
- `vl-range`
- `vl-pin`
- `vl-scrub`
- `vl-children`

- `vl-stagger`
- `vl-targets`
- `vl-depth`
- `vl-speed`
- `vl-once`
- `vl-page-transition`
- `vl-transition`

5.3 CSS variables

Use CSS custom properties as the low-level tuning layer.

Examples: - `--vl-range` - `--vl-stagger` - `--vl-motion-duration` - `--vl-motion-distance` - `--vl-ease-cinematic`

---

6. Attribute System Contract

Agents must treat the attribute system as a **declarative grammar**, not as a random set of flags.

6.1 `vl-scene`

Marks a motion-aware scene container.

Examples: - `vl-scene="hero"` - `vl-scene="story"` - `vl-scene="features"`

6.2 `vl-effect`

Declares the named effect preset.

Examples: - `vl-effect="fade-up"` - `vl-effect="flow-in"` - `vl-effect="scene-hero-reveal"`

6.3 `vl-timeline`

Declares the progress model.

Allowed v0.1 values: - `view` - `scroll` - `auto` - `state` - `hover`

6.4 `vl-range`

Defines the active progress range for timeline-driven motion.

Examples: - `vl-range="entry 0% cover 40%"` - `vl-range="entry 10% cover 60%"` - `vl-range="entry 0% exit 100%"`

6.5 `vl-pin`

Boolean attribute for pinned/sticky scene behavior.

6.6 `vl-scrub`

Boolean attribute for continuously progress-linked motion.

6.7 `vl-children`

Coordinates direct-child choreography.

Allowed v0.1 values: - `stagger` - `cascade` - `sequence`

6.8 `vl-stagger`

Defines stagger interval.

Example: - `vl-stagger="120ms"`

6.9 `vl-targets`

Targets a subset of descendants.

Examples: - `vl-targets=".vl-card"` - `vl-targets="[data-layer]"`

6.10 `vl-depth`

Declares relative motion depth.

Recommended values: - `1` - `2` - `3` - `4`

6.11 `vl-speed`

Declares speed category.

Allowed values: - `slow` - `normal` - `fast`

6.12 `vl-once`

Boolean hint for one-shot reveal behavior.

6.13 `vl-page-transition`

Page-level transition style.

Allowed values: - `fade` - `slide` - `cover` - `crossfade` - `depth`

6.14 `vl-transition`

Link-level or element-level transition style.

Allowed values: - `fade` - `slide` - `cover` - `crossfade` - `depth`

7. Preset Registry Direction

Agents should use named presets rather than inventing one-off effect systems.

Primitive effects

- `fade-in`
- `fade-up`
- `fade-down`
- `slide-left`
- `slide-right`
- `scale-in`
- `blur-in`
- `reveal-3d`
- `flow-in`

Interaction effects

- `hover-lift`
- `hover-glow`
- `underline-expand`
- `icon-shift`

State effects

- `accordion`
- `drawer`
- `panel-swap`

Scene effects

- `scene-hero-reveal`
- `scene-feature-flow`
- `scene-story-pin`
- `scene-layer-stack`

Agents must keep this registry coherent and avoid proliferating effect names without a clear need.

---

8. Primitive vs Scene Presets

This distinction is mandatory.

Primitive effect

Affects the element itself.

```
<div vl-effect="fade-up"></div>
```

Scene preset

Coordinates the element and its descendants.

```
<section vl-scene="hero" vl-effect="scene-hero-reveal"></section>
```

Agents must not blur these concepts.

---

9. Implementation Rules for Agents

9.1 Prefer variables over hardcoded values

Bad:

```
[vl-effect="fade-up"] {
  transform: translateY(28px);
  transition: all 480ms cubic-bezier(0.2, 0.8, 0.2, 1);
}
```

Better:

```
[vl-effect="fade-up"] {
  --vl-translate-y-from: 1.75rem;
  --vl-motion-duration: var(--vl-motion-normal);
  --vl-motion-ease: var(--vl-ease-cinematic);
}
```

9.2 Prefer composition over huge monolith selectors

Build small contracts that compose: - scene host - effect preset - timeline binding - child choreography - transition layer

9.3 Prefer semantic defaults

Use selectors that work naturally with semantic HTML instead of requiring wrapper inflation.

9.4 Avoid brittle magic

Do not write code that only works for a single demo unless explicitly building an isolated scene example.

9.5 Keep the playground examples premium but realistic

Examples should feel cinematic and polished, but remain maintainable and instructive.

---

10. What Agents Must Never Do

Never introduce these into core motion implementation

- GSAP
- Framer Motion
- ScrollTrigger
- Locomotive Scroll
- Barba.js
- Swup
- React as a dependency for core rendering
- Vue as a dependency for core rendering

Never replace semantic elements with div soup unnecessarily

Bad: - replacing `<button>` with clickable `<div>` - replacing `<details>` with custom accordion containers unless there is a compelling reason

Never bypass the attribute system casually

Do not implement bespoke scene logic when a general `vl-*` contract should exist.

Never hardcode brand or demo-specific values into the framework core

Framework core must remain reusable.

Never ignore reduced motion

Every substantial animation pattern must have a reduced-motion fallback or disable path.

---

11. Accessibility Requirements

Every agent-generated feature must consider:

- semantic tags
- keyboard interaction
- visible focus states
- readable contrast
- reduced-motion behavior
- non-blocking content flow

Global reduced motion baseline

```
@media (prefers-reduced-motion: reduce) {
  [vl-effect],
  [vl-scene],
  [vl-transition],
  [vl-page-transition] {
    animation: none !important;
    transition-duration: 0ms !important;
  }
}
```

Agents may refine this, but must not omit reduced-motion handling.

---

12. Documentation Expectations

When agents create docs pages, they should include:

- purpose
- markup example
- class/attribute API
- CSS variable hooks
- states and variants
- accessibility notes
- browser support notes
- implementation notes
- live demo intent

Docs are not secondary. Docs are part of the product.

---

13. Suggested Build Order

Agents should generally work in this order unless directed otherwise.

Phase 1 — Foundation

1. reset
2. tokens
3. typography
4. color system
5. spacing/radius/shadows

Phase 2 — Layout primitives

1. container
2. stack
3. grid

4. split layout
5. app shell
6. sticky scene wrapper

Phase 3 — Motion primitives

1. `vl-effect`
2. `vl-timeline`
3. `vl-range`
4. `fade-up`, `fade-in`, `flow-in`, `reveal-3d`
5. hover presets

Phase 4 — Choreography

1. `vl-children`
2. `vl-stagger`
3. nth-child stagger helpers
4. scene presets

Phase 5 — Scenes

1. hero reveal scene
2. feature flow scene
3. pinned story scene
4. layered media scene

Phase 6 — Transitions

1. page transition layer
2. view transition support
3. shared element examples

Phase 7 — Components

1. buttons
2. cards
3. nav
4. fields
5. accordion
6. dialog/drawer
7. tabs

Phase 8 — Docs and examples

1. playground demos
2. docs pages
3. starter scenes
4. premium examples

---

14. Safe Prompting Patterns for Agents

Below are recommended prompt styles for Cursor and Copilot.

14.1 When asking for framework code

Use prompts like:

> Implement `vl-effect="fade-up"` and `vl-timeline="view"` in `packages/css` using layered CSS only. Do not use JavaScript. Preserve the existing namespace and token system. Keep the implementation reusable, documented, and reduced-motion aware.

14.2 When asking for new scene presets

> Create a new scene preset called `scene-feature-flow` that coordinates heading, copy, and cards using scroll-driven CSS. Use `vl-scene`, `vl-effect`, `vl-children`, and CSS custom properties. Avoid demo-specific hardcoding.

14.3 When asking for docs

> Create a docs page for `vl-effect` with API explanation, allowed values, implementation notes, accessibility notes, and two HTML examples using only semantic markup.

14.4 When asking for refactors

> Refactor this implementation so that public behavior is expressed through `vl-*` attributes and tokenized CSS variables rather than one-off component selectors.

14.5 When asking for review

> Review this Velora code for violations of project rules: no JS animation logic, semantic HTML, namespace correctness, reduced-motion support, token usage, and layer organization.

---

15. GitHub Copilot Guidance

GitHub Copilot works best when given short, precise, local instructions.

Recommended usage

- use focused inline comments before generating code
- keep scope narrow per file
- mention exact constraints in comments
- ask for refactors into tokens/variables after initial output

Good Copilot comment examples

```
/* Implement a Velora fade-up effect preset using vl-effect and vl-
timeline=view. No JS. Use CSS custom properties and reduced-motion support.
*/
```

```
// Create a minimal theme persistence module for Velora. No animation logic.
Keep bundle tiny and framework-agnostic.
```

Copilot review checklist

After accepting output, verify: - no forbidden libraries - no unnecessary wrappers - no hardcoded magic values where tokens should exist - no broken namespace - no accessibility regressions - no JavaScript added for motion that CSS can do

---

16. Cursor Guidance

Cursor works best when used for broader refactors, multi-file generation, and architecture-aware edits.

Recommended usage

Use Cursor for: - creating or refactoring layered CSS files - generating docs pages from framework contracts - implementing a new preset across CSS + playground + docs - reviewing project-wide consistency - generating starter scenes with strict API rules

Strong Cursor prompt example

> You are working on Velora, a motion-native frontend platform. Follow these rules strictly: no React/Vue/GSAP/Framer Motion, no JS animation logic, semantic HTML only, CSS layers required, namespace `.vl-`, `--vl-`, and `vl-*`. Implement `vl-children="stagger"` and `vl-stagger` in the CSS package, update the playground with one premium example, and add a docs draft explaining the API and reduced-motion behavior.

Another Cursor prompt example

> Audit the current repository for anything that violates Velora architecture. Report issues grouped by: naming, CSS layers, token misuse, accessibility, JS overreach, and missing docs. Then propose a minimal patch plan.

Cursor review checklist

Before applying large edits, verify: - public API remains small - code is not demo-only unless intentionally in playground - presets are named consistently - scene presets and primitive effects are not mixed up - docs match real implementation

---

17. Suggested Agent Workflows

Workflow A — New effect preset

1. define preset name
2. add token hooks if needed
3. implement in `velora.motion`
4. add reduced-motion rule
5. add playground example
6. add docs example

Workflow B — New scene preset

1. define the scene contract
2. identify participating descendants
3. implement scene-level CSS variables
4. add child choreography rules
5. test semantic HTML version
6. document expected markup shape

Workflow C — New layout primitive

1. define class API
2. map to container-query-friendly structure
3. add token hooks
4. test in docs/playground
5. ensure composition with motion attributes

Workflow D — Refactor pass

1. find hardcoded values
2. convert to tokens
3. replace ad hoc selectors with public contracts
4. move rules into correct layers
5. add docs notes if behavior changes

---

18. Quality Bar

Every accepted implementation should aim for:

- semantic correctness
- CSS elegance
- readable source
- small public API surface
- polished visual result
- strong progressive enhancement
- maintainable documentation

Velora should feel premium both in the browser and in the source code.

---

19. Definition of Done

A Velora feature is done when:

1. the public API is clear
2. namespaces are correct
3. tokens are used appropriately
4. reduced-motion behavior exists
5. docs can explain it simply
6. the playground demonstrates it cleanly
7. implementation fits the architecture
8. no forbidden JS framework/runtime pattern was introduced

---

20. Final Instruction to Agents

When in doubt, choose the path that is:

- more semantic
- more CSS-native
- more tokenized
- more composable
- more accessible
- easier to document
- less dependent on JavaScript

Velora should prove that modern HTML and CSS are enough to build premium motion systems when the architecture is designed correctly.